

Q1. Harshad number

ANSWER

```
n = int(input("Enter a number: "))

def is_harshad(num):
    s = sum(int(digit) for digit in str(num))
    return num % s == 0

if n > 0 and is_harshad(n):
    print(f"{n} is a Harshad number.")
else:
    print(f"{n} is NOT a Harshad number.")

print("Harshad numbers between 1 and 500 are:")
for i in range(1, 501):
    if is_harshad(i):
        print(i, end=" ")
```

Q2. GCD

CODE

```
a = int(input("Enter first: "))
b = int(input("Enter second: "))

while b != 0:
    temp = a
    a = b
    b = temp % a

print("GCD is", a)
```

INPUT

48
18

OUTPUT

Enter first: Enter second: GCD is 6

Q3. sum to a target value

ANSWER

```
def find_pairs(lst, target):
    seen = set()
    pairs = []
    for num in lst:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
            seen.add(num)
    return pairs

data = [int(x) for x in input("Enter numbers: ").split()]
t = int(input("Target: "))
result = find_pairs(data, t)
print("Pairs:", result)
```

Q4. Bitwise Operators

ANSWER

```
# Bitwise Operators - Trace and exact output

a = 0b1101
b = 0o25
c = 0x1A
result = (a & b) | c

print(f"a={a}, b={b}, c={c}, result={result}")
print(bin(result), oct(result), hex(result))
print(result >> 2, result << 1, ~result)

# Data types after assignment
print("Data types:")
print("type(a):", type(a))
print("type(b):", type(b))
print("type(c):", type(c))
print("type(result):", type(result))
```

Q5. binary search

CODE

```
def rotated_binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid

        # Left half is sorted
        if arr[low] <= arr[mid]:
            if arr[low] <= target < arr[mid]:
                high = mid - 1
            else:
                low = mid + 1
        # Right half is sorted
        else:
            if arr[mid] < target <= arr[high]:
                low = mid + 1
            else:
                high = mid - 1

    return -1

data = [4, 5, 6, 7, 0, 1, 2]
t = int(input("Target: "))
result = rotated_binary_search(data, t)
print("Found at:", result)
```

OUTPUT

Target: Found at: 4